Beyond Code: An SRE Agent for Self-Healing Operations

Marc Mueller

4tecture©
empower your software solutions



About me:

Marc Müller
Principal Consultant
@muellermarc

MVP Microsoft Most Valuable Professional

4tecture©
empower your software solutions

Our Products:

Multi-Tenant OpenID Connect Identity Provider

Enterprise Application Framework for .NET

ProAuth

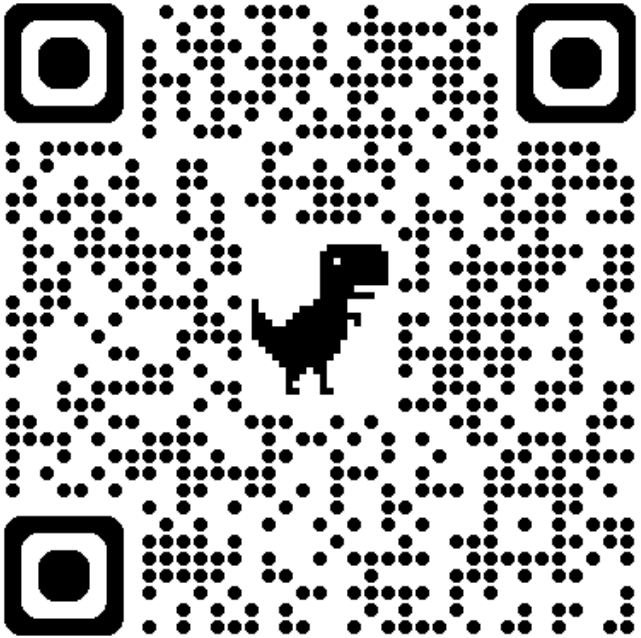
ReaFx

www.proauth.net

www.reafx.net

Slide Download

https://www.4tecture.ch/events/dwx26-sre-agent



Agenda

- First demo: pods red, agent fixes
- Second demo: metrics reveal hidden failure
- Why build our own SRE agent?
- Architecture for Agentic Operations
- Demo code, step by step
- Production guardrails

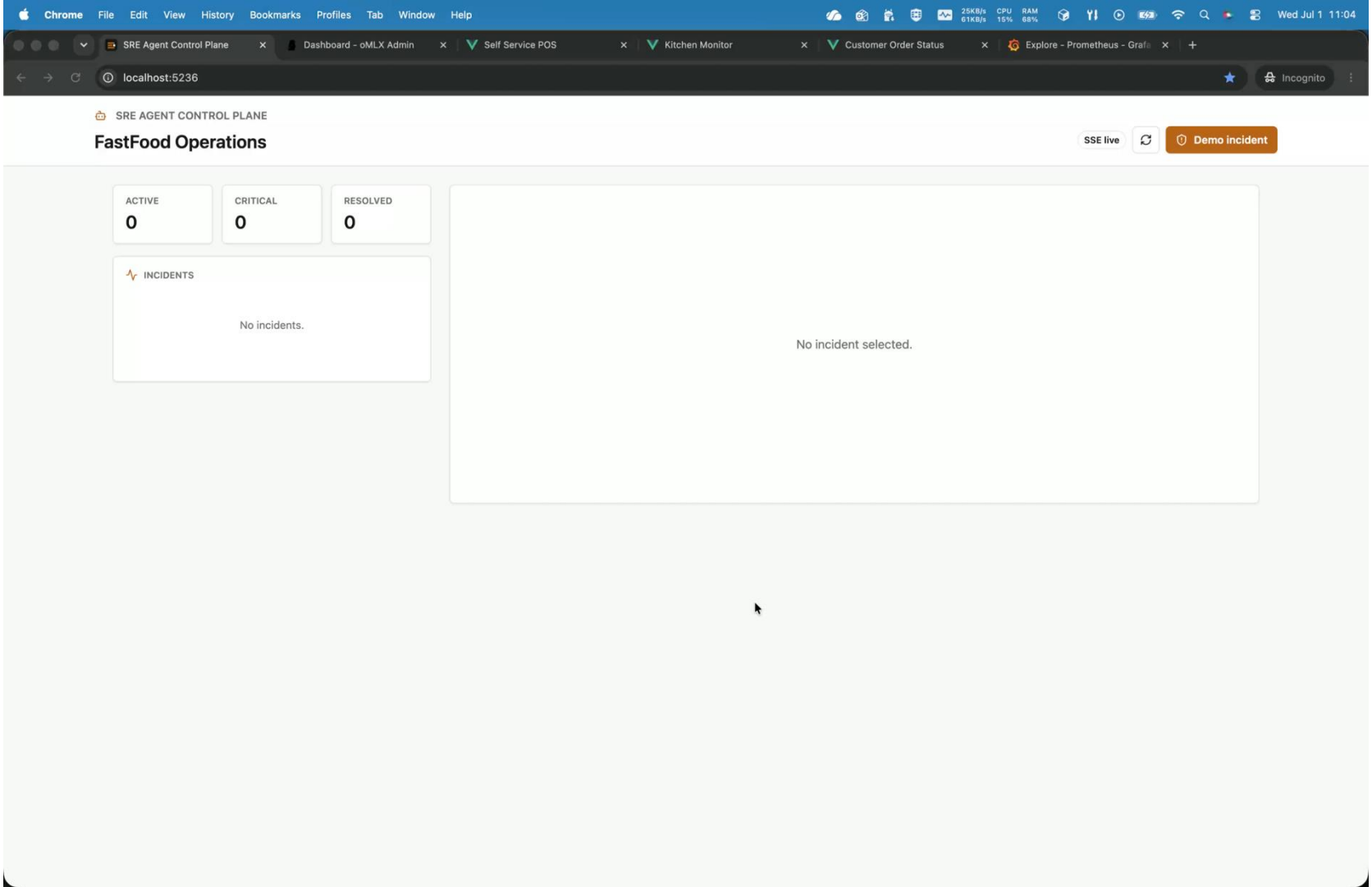
Cold Open: Pods Turn Red

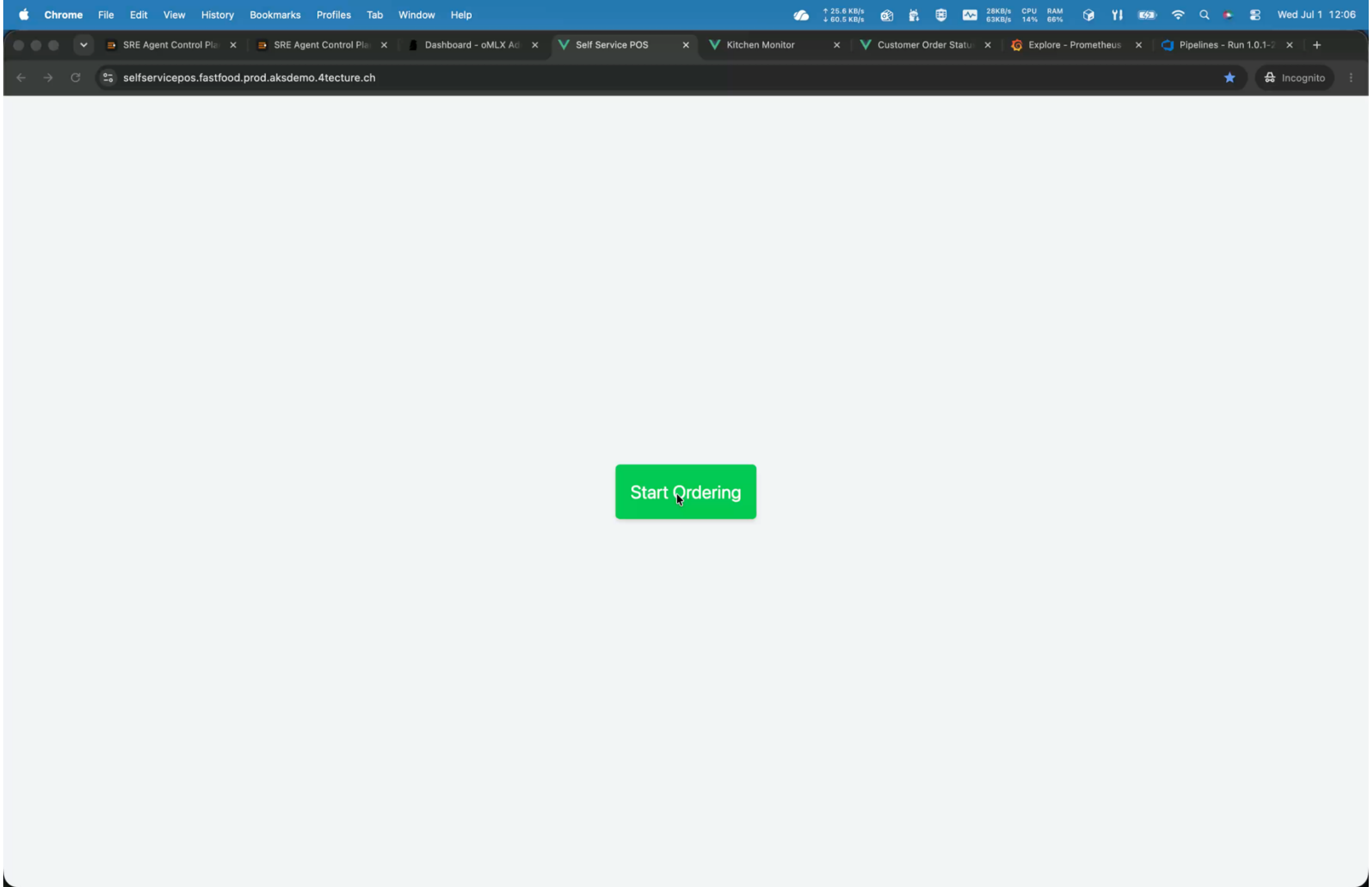
- Inject a FastFood deployment error
- Kubernetes reports unhealthy pods
- SRE agent starts automatically
- Agent analyzes, patches, verifies

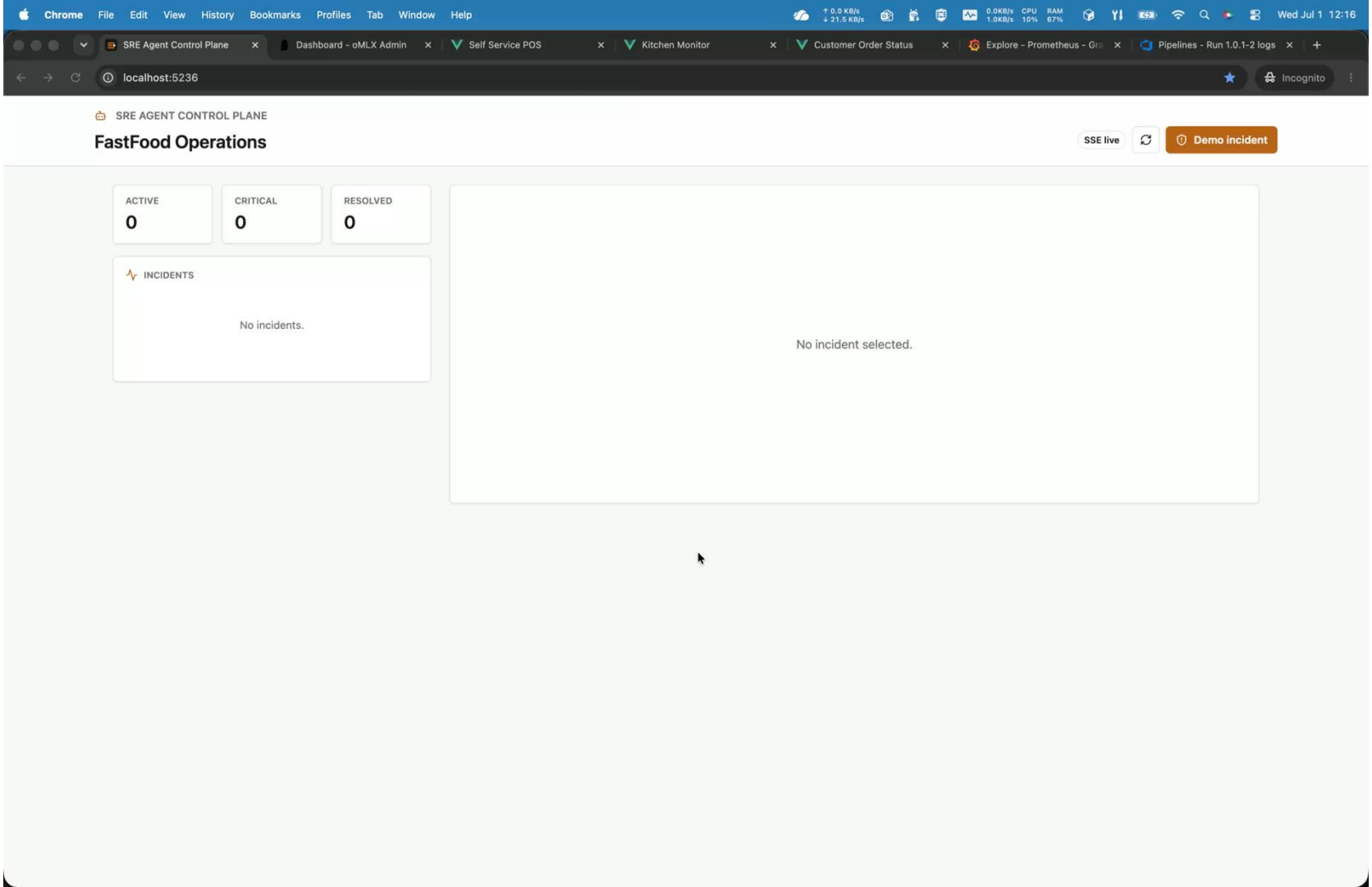
Demo



Pods go red


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Wed Jul 1 11:04
SRE Agent Control Plane
Dashboard - oMLX Admin
Self Service POS
Kitchen Monitor
Customer Order Status
Explore - Prometheus - Grafa
localhost:5236
Incognito
SRE AGENT CONTROL PLANE
FastFood Operations
SSE live
Demo incident
ACTIVE
0
CRITICAL
0
RESOLVED
0
INCIDENTS
No incidents.
No incident selected.


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Wed Jul 1 12:06
SRE Agent Control Pla
SRE Agent Control Pla
Dashboard - oMLX Ad
Self Service POS
Kitchen Monitor
Customer Order Statu
Explore - Prometheus
Pipelines - Run 1.0.1-2
selfservicepos.fastfood.prod.aksdemo.4tecture.ch
Incognito
Start Ordering


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Wed Jul 1 12:16
SRE Agent Control Plane
Dashboard - oMLX Admin
Self Service POS
Kitchen Monitor
Customer Order Status
Pipelines - Run 1.0.1-2 logs
localhost:5236
Incognito
SRE AGENT CONTROL PLANE
FastFood Operations
SSE live
Demo incident
ACTIVE
0
CRITICAL
0
RESOLVED
0
INCIDENTS
No incidents.
No incident selected.

SRE AGENT CONTROL PLANE

FastFood Operations

SSE live | Demo incident

ACTIVE	CRITICAL	RESOLVED
0	1	1

INCIDENTS

Kitchen order intake stalled while paid... Resolved

prod / kitchenservice

Grafana detected paid orders in OrderService without corresponding kitchen order intake. Pods may still be healthy...

Critical | Resolved | fastfood-demo

RUN MODE AutoRemediate | Run auto-remediation

Kitchen order intake stalled while paid orders continue

Grafana detected paid orders in OrderService without corresponding kitchen order intake. Pods may still be healthy, so investigate Dapr pub/sub routing, component scopes, subscriptions, and delivery configuration.

CONFIGURED INCIDENT MODE	EFFECTIVE WRITE POLICY
AutoRemediate	Agent may apply safe fixes automatically.

AUTO-RUN ON NEW INCIDENTS	RUNTIME
Enabled	Copilot / Qwen3.6-35B-A3B-8bit

NAMESPACE	SERVICE	SOURCE	UPDATED
prod	kitchenservice	grafana	12:24:00 PM

AGENT SESSION

```
2. **Dapr sidecar log** (new pod) confirms subscription:
   ```
   app is subscribed to the following topics:
   [deadletter_kitchenitemfinished deadletter_kitchenorderstartprocessing
    deadletter_orderpaid orderpaid] through pubsub=pubsub
   ```
3. **All 6 pods** 2/2 Running, no restarts on new pod.
4. **Dapr initialized** successfully in 338ms with no component errors.

## Post-incident

- **Prevention:** Add CI/CD validation that the pubsub component scopes include all
FastFood app IDs (`orderservice`, `kitchenservice`, `financeservice`,
`orderserviceactors`, `frontendselfservicepos`, `frontendkitchenmonitor`,
`frontendcustomerorderstatus`) as a compliance gate on any Dapr component changes.
- **Observability gap:** The `kitchenservice_orders_created_total` metric is currently
reactive (polling). Consider adding a pub/sub subscription confirmation alert that fires
when a Dapr sidecar fails to register on expected topics.
```

ACTIONS

No tool actions recorded.

REPORT

Open printable report

INCIDENT REPORT

Kitchen order intake stalled while paid orders continue

Print

Kitchen Order Intake Stalled — Missing Pub/Sub Scope

When: 2026-07-01T10:22:00Z
Cluster: fastfood-demo
Namespace: prod
Impact: Paid orders flowing through OrderService were not reaching KitchenService. Customers saw orders confirmed/paid but kitchens never received intake. Up to 2 orders missing per 5-minute window.

Signals

Grafana SLO alert `fastfood-kitchen-intake-slo` fired:

- `paid-orders-without-kitchen-intake` : **2.5** (threshold ≥ 2)
- `paid-orders-last-5m` : **2.5** (orders were being paid)
- `kitchen-orders-created-last-5m` : **0** (kitchen intake was zero)
- `paid-orders-last-15m` : **6.43** (6 paid orders in 15m)
- `kitchen-orders-created-last-15m` : **6.43** (previously healthy — recovery expected after fix)

Diagnosis

All 6 pods were healthy (2/2 Running). Dapr sidecar logs on `kitchenservice` showed only placement table updates and `GetPendingOrders` polling — no pub/sub event delivery.

Inspected the Dapr `pubsub` component and found **scope drift**: `kitchenservice` was missing from `spec.scopes`.

Before (broken):

```
scopes:
- orderservice
- financeservice
- orderserviceactors
- frontendselfservicepos
- frontendkitchenmonitor
- frontendcustomerorderstatus
```

Root cause: `kitchenservice` was not listed in the pubsub component scopes, so the Dapr sidecar never subscribed it to the `orderpaid` topic. Orderservice published `orderpaid` events, but kitchenservice's daprd silently dropped them because the app ID was not authorized on the component.

Actions Taken

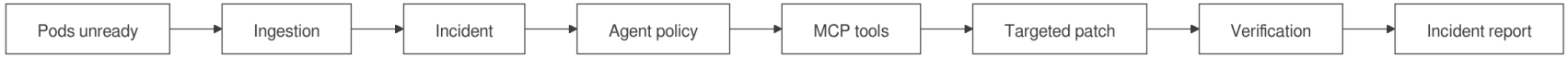
Step	Action	Result
1	`kubectl patch component pubsub -n prod --type=merge` to add `kitchenservice` to scopes	Patch rejected (generation stayed 54). Root field `scopes` was misidentified as `spec.scopes`.
2	`kubectl patch component pubsub -n prod --type=merge` — corrected path: `scopes` (root-level field)	**Success** — generation 54→55. `kitchenservice` added.
3	`kubectl rollout restart deployment/kitchenservice -n prod`	Pod `kitchenservice-79f79864bf-6ttk6` came up in 10s.

Exact patch applied:

```
{
  "scopes": [
    "orderservice",
    "financeservice",
    "orderserviceactors",
    "frontendselfservicepos",
    "frontendkitchenmonitor",
    "frontendcustomerorderstatus",
    "kitchenservice"
  ]
}
```

Unaffected fields: All `spec.metadata` entries (redisHost, redisPassword, redisDB, maxLenApprox), `spec.type`, `spec.version`, labels, annotations, creationTimestamp, uid — all preserved.

What Just Happened



Why This Matters

- Operations often becomes toil
- Diagnosis creates context switching
- Runbooks quietly go stale
- Good fixes require evidence
- Repetition calls for automation

AGENTIC OPERATIONS

From Chatbot To Operator

- Chat: answers questions
- Assistant: suggests next steps
- Agent: uses tools autonomously
- Operator: acts within policy
- SRE agent: evidence-first, recovery-focused

Autonomy

Lower

- Read Only
- Suggest Fix
- Postmortem Only

Higher

- Approval Required
- Auto Remediate
- Policy-gated writes

The Evidence Contract

- Read live state
- Ground hypotheses in sources
- Plan the smallest safe change
- Document before and after
- Verify recovery actively

Why Not Just Buy It?

- There are SRE agent services available for Azure and other clouds
- Customized agent gives more control over signals, tools, and policies
- We are building it to learn what is actually required for a safe, evidence-first SRE agent

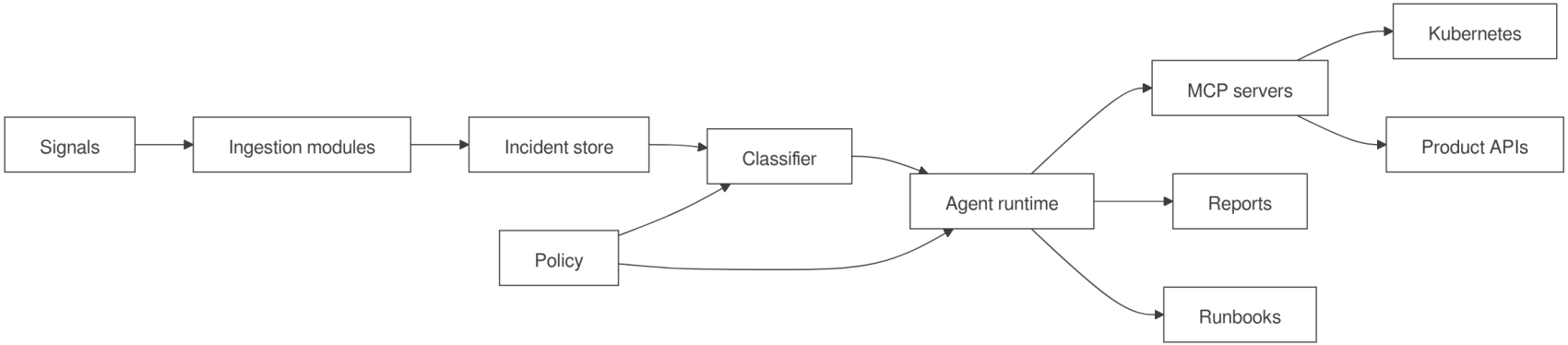
Azure SRE Agent

- Queries Application Insights and identifies a memory trend that started 40 minutes before the alert
- Correlates the trend with a deployment event from your GitHub repository two hours earlier
- Identifies the specific commit and proposes two mitigations: restart the affected pod and adjust the memory scaling threshold (HPA)
- Creates a ticket in ServiceNow, PagerDuty, or your incident channel with the full investigation summary prefilled

Source: https://learn.microsoft.com/azure/sre-agent/overview

ARCHITECTURE

Reference Architecture



Ingestion: Where Incidents Start

- Kubernetes pod health
- Prometheus or Grafana alerts
- Log and trace anomalies
- ChatOps or manual escalation
- Scheduled health checks

Incident Core

- Receive signal
- Deduplicate fingerprint
- Apply rule-based triage
- Build mode-specific policy
- Write timeline and events

Tools: MCP As The Operations Interface

- Standardized tool descriptions
- Servers encapsulate target systems
- Agent discovers available tools
- Policies constrain tool usage
- Audit per tool call

Skills Are Runbooks For Agents

- Version product knowledge
- Block dangerous shortcuts
- Encode known failure modes
- Require explicit verification
- Review them like code

MCP Server For Knowledge Base Access

- MCP Server for internal knowledge base
- Advanced search and retrieval
- Skills can query KB for context

DEMO CODE

Repository Map

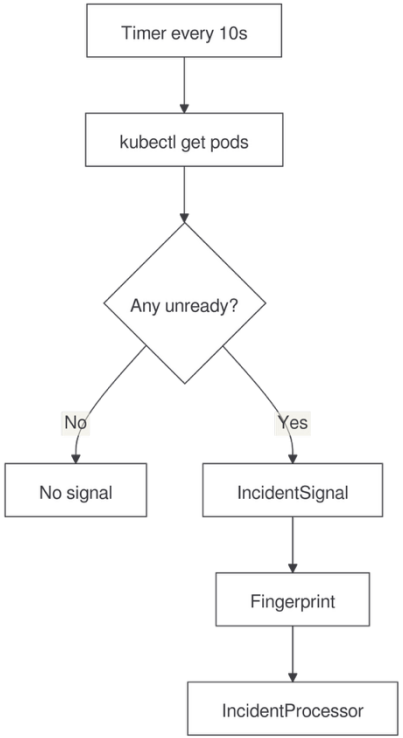
- `SreAgent.Core`: incidents, policies, ingestion
- `SreAgent.Agent.Copilot`: runtime adapter
- `SreAgent.Web`: dashboard and prompts
- `mcp-setup/k8s`: ServiceAccount and RBAC
- `skills/`: operational runbooks

Demo

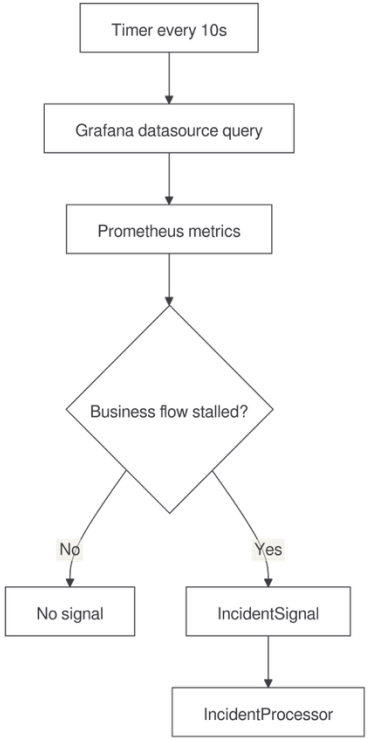


How it was built

Kubernetes Ingestion



Grafana Ingestion



Demo 2: Pods Green, Flow Broken

- Misconfigure Dapr pub/sub scopes
- Orders can still be paid
- Kitchen receives no order events
- Pods remain Ready
- Grafana emits the signal

Trigger Query

```
(
  (sum(increase(orderservice_orders_paid_total[5m])) or vector(0)) >= 2
)
and
(
  (sum(increase(kitchenservice_orders_created_total[5m])) or vector(0)) == 0
)
```

Evidence Queries

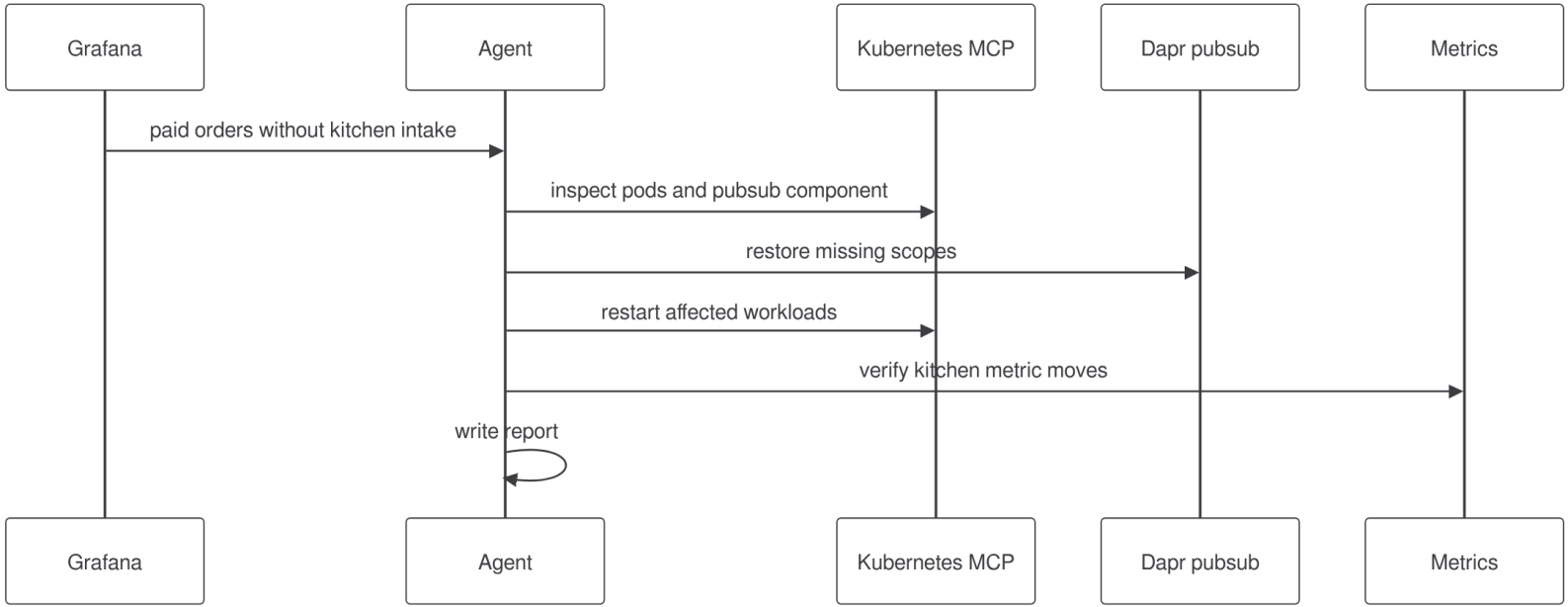
```
sum(increase(orderservice_orders_paid_total
[5m])) or vector(0)
sum(increase(kitchenservice_orders_created_
total[5m])) or vector(0)

sum(increase(orderservice_orders_paid_total
[15m])) or vector(0)
sum(increase(kitchenservice_orders_created_
total[15m])) or vector(0)
```

Telemetry Is The Prerequisite

- Agent needs actionable signals
- Metrics encode expected behavior
- Logs explain local failures
- Traces connect service boundaries
- Bad telemetry means blind automation

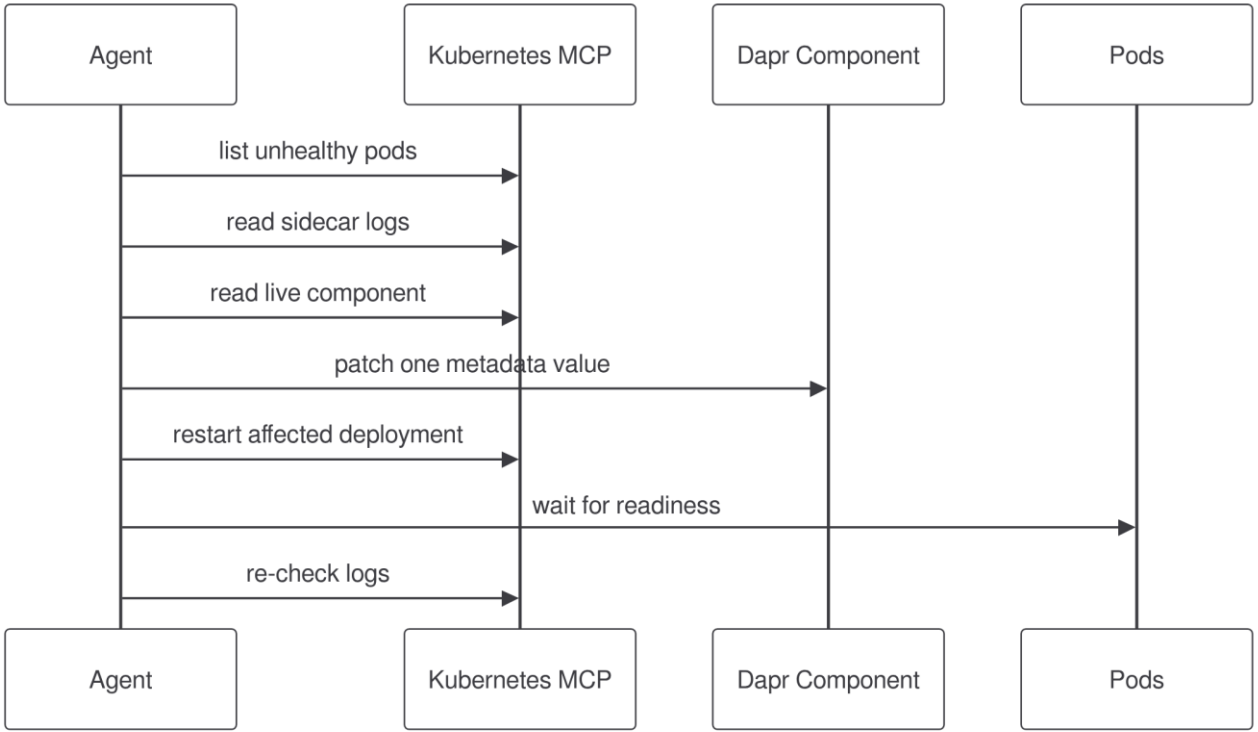
Demo 2 Remediation Sequence



Prompt: The Operating Contract

- Restore service safely
- Prefer tools over guesses
- Read before changing
- Patch minimally
- Verify readiness and logs
- Produce incident report

Remediation Sequence in our Demo



Incident Report As Artifact

- Signals
- Diagnosis
- Actions Taken
- Verification
- Post-incident

GUARDRAILS

Least Privilege Service Account

- Dedicated ServiceAccount
- RoleBindings per namespace
- Only required resources
- Only required verbs
- No cluster-admin shortcut

Tool Approval Is Policy

- Allowlist MCP servers
- Denylist risky tools
- Require approval before writes
- Sandbox host access
- Log every tool call

Prompt Injection Is Operational Reality

- Logs contain untrusted input
- Tickets can contain instructions
- Alert payloads are data, not commands
- Tools need hard boundaries
- Reports need sources and evidence

Self-Healing Requires Design

- Redundancy reduces risk
- Bulkheads limit blast radius
- Automatic actions must be idempotent
- Rollback must be known first
- Recovery must be tested

Decision Rule For Auto-Fix

- Is the symptom unambiguous?
- Is root cause evidenced?
- Is the fix small and reversible?
- Are permissions minimal?
- Is verification automatable?

RECAP

Takeaways

- SRE agents need more than a model
- Evidence beats confidence
- MCP makes tools portable
- Skills make runbooks actionable
- Autonomy should grow gradually
- Guardrails are product features

Thank you! Let’s continue the conversation

If you would like to discuss your challenges or explore how we can support you, please contact us.

4tecture GmbH
Industriestrasse 25
CH-8604 Volketswil
www.4tecture.ch

Marc Müller
Principal Consultant

4tecture©
empower your software solutions

ProAuth

ReaFx

References

- Google SRE Book - Eliminating Toil: https://sre.google/sre-book/eliminating-toil/
- Microsoft Learn - Azure SRE Agent: https://learn.microsoft.com/azure/sre-agent/overview
- Microsoft Learn - Agent Framework: https://learn.microsoft.com/agent-framework/overview/
- Model Context Protocol: https://modelcontextprotocol.io/docs/getting-started/intro
- Kubernetes Docs - RBAC: https://kubernetes.io/docs/reference/access-authn-authz/rbac/
- Azure Monitor Action Groups: https://learn.microsoft.com/azure/azure-monitor/alerts/action-groups
- Prometheus Docs - Querying Functions: https://prometheus.io/docs/prometheus/latest/querying/functions/
- Prometheus Docs - Querying Operators: https://prometheus.io/docs/prometheus/latest/querying/operators/
- Grafana Docs - Prometheus Data Source: https://grafana.com/docs/grafana/latest/datasources/prometheus/
- Azure Well-Architected - Self-healing: https://learn.microsoft.com/azure/well-architected/reliability/self-preservation
- GitHub Docs - Copilot CLI: https://docs.github.com/en/copilot/concepts/agents/copilot-cli/about-copilot-cli
- OWASP GenAI Security: https://genai.owasp.org/llm-top-10/